



Operaciones

Operaciones de conjuntos

Cuando trabajamos con información espacial, como polígonos, es común tener que analizar sus relaciones espaciales. En muchos casos, es importante determinar si se superponen y, en otros, generar nuevas formas a partir de sus intersecciones o diferencias.

Por nombrar algunos ejemplos:

- La unión de los polígonos que representan los estados de la República Mexicana nos daría como resultado la superficie total del país.
- La intersección de las rutas de dos líneas de transporte podría revelar puntos estratégicos para realizar cambios de línea.
- En un entorno empresarial, podríamos calcular el espacio disponible en una nave industrial restando el área ocupada por oficinas de la superficie total de la nave.

A continuación, exploraremos las cuatro operaciones de conjuntos más utilizadas en el análisis geoespacial, las cuales te permitirán manipular y combinar datos espaciales de manera efectiva.

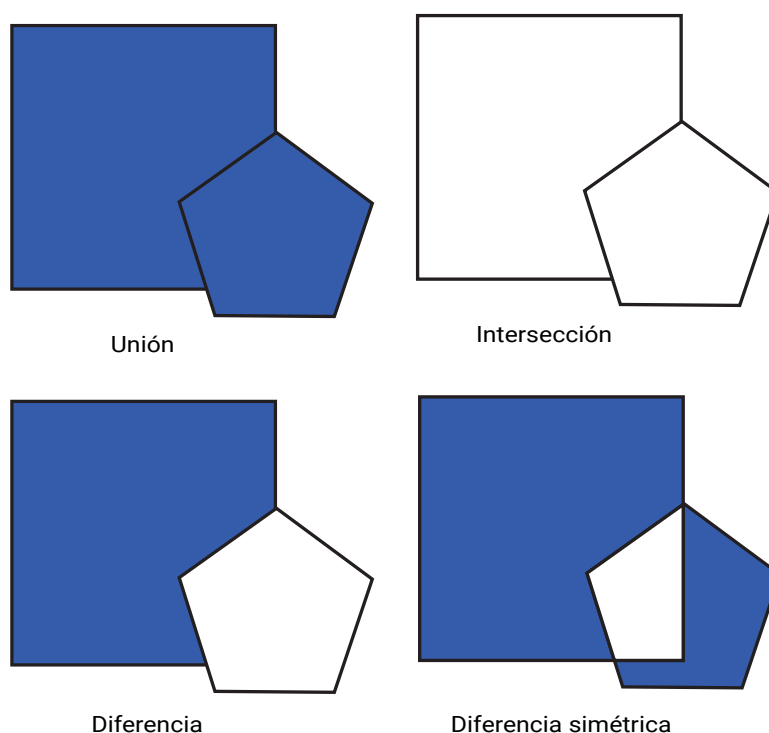


Figura 1. Operaciones comunes en GeoPandas.

En GeoPandas, las operaciones de conjuntos se realizan mediante la función `overlay`. Es importante destacar que estas operaciones se aplican a nivel de GeoDataFrames, no a geometrías individuales. Esto significa que cada geometría del primer GeoDataFrame se comparará con cada geometría del segundo, generando un nuevo conjunto de datos basado en la operación seleccionada.

Para comprender mejor cómo funciona `overlay`, comenzaremos con un ejemplo utilizando polígonos simples. Esto nos permitirá visualizar los resultados de cada operación antes de aplicarlas a datos reales en un caso práctico.

Antes de comenzar, crearemos algunos polígonos y los almacenaremos en dos GeoDataFrames, que servirán como base para nuestras pruebas. A continuación, te mostramos el código para generarlos:

```
# Instalar geopandas y rtree para usar overlay
!pip install geopandas
!apt install libspatialindex-dev
!pip install rtree

# Importar todas las librerías con las que vamos a trabajar
import geopandas
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
# Importar la clase Polígono
from shapely.geometry import Polygon

# Creamos nuestros polígonos a partir de los puntos que
forman los vértices
cuadrados = geopandas.GeoSeries([Polygon([(1,1), (1,3),
```

```

(4,3), (4,1)]), Polygon([(2,5), (2,7), (5,7), (5,5)]))
triangulos = geopandas.GeoSeries([Polygon([(2,2), (4,5),
(6,2)]), Polygon([(4,6), (6,9), (6,6)])])

# Creamos los dos GeoDataFrames
df_cuadrados = geopandas.GeoDataFrame({'geometry':
cuadrados, 'df_cuadrados':[1,2]})
df_triangulos = geopandas.GeoDataFrame({'geometry':
triangulos, 'df_triangulos':[1,2]})

# Dibujamos los contornos de cuadrados en azul y de
triángulos en verde
ax = df_cuadrados.boundary.plot(color='blue')
df_triangulos.boundary.plot(ax=ax, color='green')

```

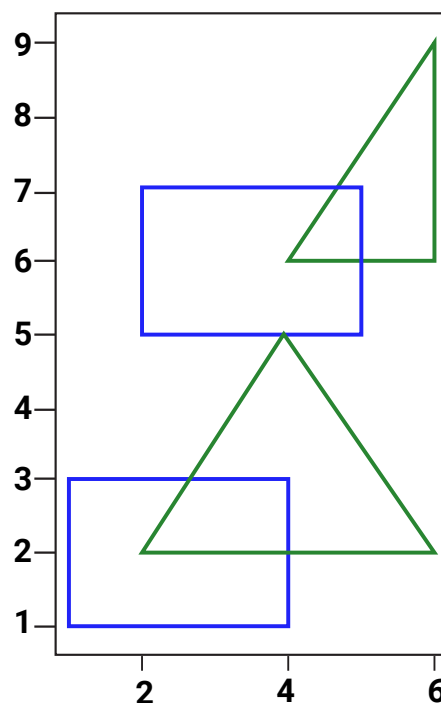


Figura 2. Ejemplo de visualización de GeoDataFrames con varios polígonos.

Unión (union)

La función `overlay` en GeoPandas permite combinar dos GeoDataFrames determinando el conjunto de todas las geometrías resultantes de su superposición. En el caso de la **operación de `union`**, **el resultado incluirá todas las geometrías de ambos conjuntos, manteniendo su estructura original y cualquier área de intersección.**

En el siguiente ejemplo, veremos cómo aplicar la operación `union` utilizando la función `overlay`. Para ello, es fundamental especificar el parámetro `how="union"`, que indica que queremos conservar todas las geometrías involucradas en la superposición.

```
# Obtenemos la unión usando overlay
union = geopandas.overlay(df_cuadrados, df_triangulos,
                           how='union')
ax = union.plot(alpha=0.5, cmap='rainbow')
# Graficamos los contornos de las geometrías originales
df_cuadrados.boundary.plot(ax=ax, color='black')
df_triangulos.boundary.plot(ax=ax, color='black')
```

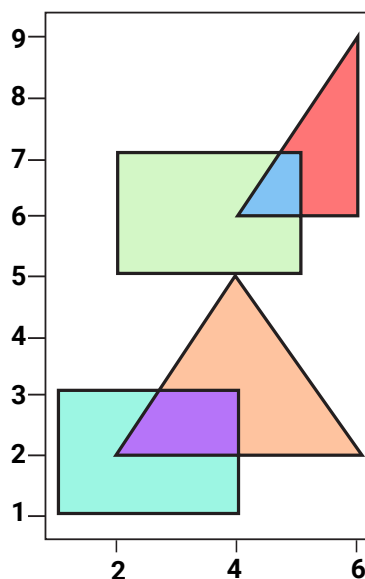


Figura 3. Visualización de una unión de GeoDataFrames.

Es posible que te preguntes: ¿cuántas geometrías componen el resultado de la operación? Para facilitar su identificación y conteo, hemos identificado cada una con un color diferente en la visualización. En este caso, el resultado incluye seis geometrías distintas, representadas en verde, azul, rojo, turquesa, morado y naranja.

Además, hemos añadido el contorno de las geometrías originales para que puedas comparar la transformación y comprender mejor cómo se generó el nuevo conjunto de datos.



Intersección (intersection)

La **operación de intersección** devuelve únicamente las geometrías resultantes de la superposición entre los dos GeoDataFrames. Es decir, **el resultado estará compuesto sólo por las áreas donde ambas capas comparten espacio**.

En el siguiente ejemplo, aplicamos la función `overlay` de GeoPandas para realizar la intersección. Para ello, es fundamental utilizar el parámetro `how="intersection"`, que filtra y conserva únicamente las geometrías superpuestas.



```
# Obtenemos la intersección usando overlay
interseccion = geopandas.overlay(df_cuadrados, df_triangulos,
how='intersection')
ax = interseccion.plot(alpha=0.5, cmap='rainbow')
# Graficamos los contornos de las geometrías originales
df_cuadrados.boundary.plot(ax=ax, color='black')
df_triangulos.boundary.plot(ax=ax, color='black')
```

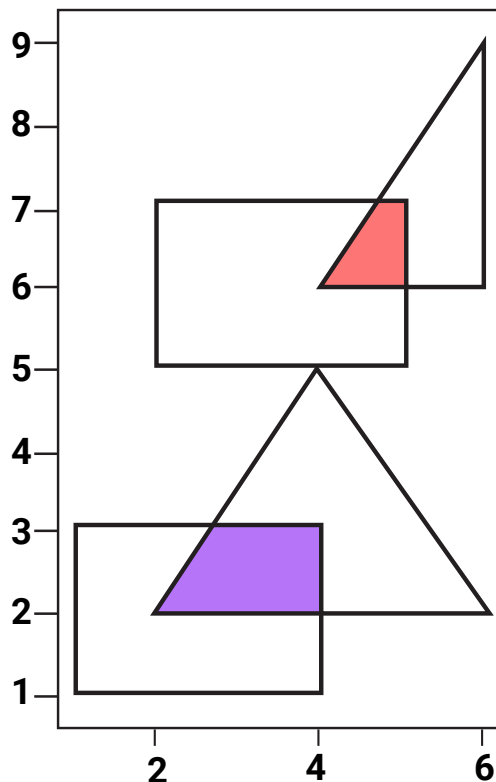


Figura 4. Visualización de una intersección de GeoDataFrames.

En este caso, el resultado consta de dos geometrías, representadas en rojo y morado en la visualización. Además, hemos dibujado el contorno de las geometrías originales para facilitar la comparación. Es importante destacar que las áreas en blanco no forman parte de la intersección y, por lo tanto, no se incluyen en el resultado.

Diferencia (difference)

La **operación de diferencia** genera un nuevo conjunto de geometrías eliminando del primer GeoDataFrame las áreas en las que se intersecta con el segundo. En otras palabras, **el resultado conserva únicamente las partes de las geometrías originales que no comparten espacio con la segunda capa.**

En el siguiente ejemplo, aplicamos la función `overlay()` de GeoPandas para calcular la diferencia. Para ello, es fundamental utilizar el parámetro `how="difference"`, que extrae únicamente las áreas que no se superponen.

```
# Obtenemos la diferencia usando overlay
diferencia = geopandas.overlay(df_cuadrados, df_triangulos,
                                how='difference')
ax = diferencia.plot(alpha=0.5, cmap='rainbow')
# Graficamos los contornos de las geometrías originales
df_cuadrados.boundary.plot(ax=ax, color='black')
df_triangulos.boundary.plot(ax=ax, color='black')
```

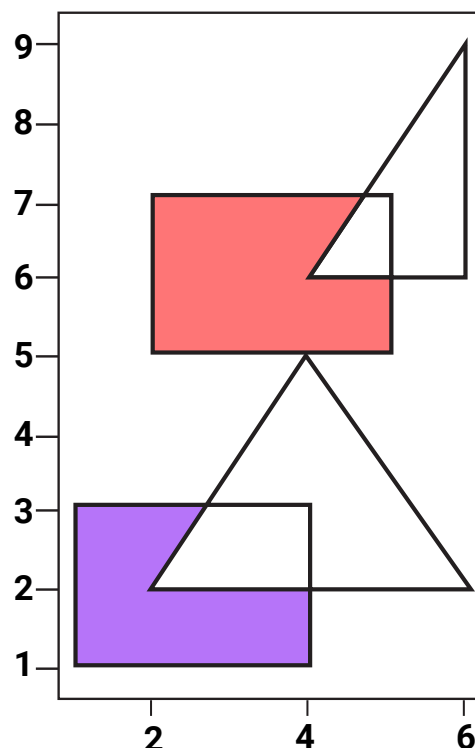


Figura 5. Visualización de una diferencia de GeoDataFrames.

En este caso, el resultado consta de dos geometrías, representadas con los colores rojo y morado. Sin embargo, a diferencia de la intersección, aquí el resultado corresponde a la “resta” del segundo GeoDataFrame sobre el primero.

Para facilitar su interpretación, hemos añadido el contorno de las geometrías originales en la gráfica. Es importante recordar que todas las áreas en blanco no forman parte del resultado de la diferencia.



Diferencia Simétrica (`symmetric difference`)

La operación de **diferencia simétrica** elimina las áreas de intersección entre ambos GeoDataFrames, conservando únicamente las geometrías que no comparten espacio. Es decir, **el resultado contiene todas las partes de las geometrías originales, excepto aquellas en las que los dos conjuntos se superponen.**

En el siguiente ejemplo, utilizamos la función `overlay()` de GeoPandas para calcular la diferencia simétrica. Para ello, es fundamental establecer el parámetro `how="symmetric_difference"`, que elimina las zonas de intersección y conserva el resto de las geometrías.

```
# Obtenemos la diferencia simétrica usando overlay
diferencia_sim = geopandas.overlay(df_cuadrados, df_
triangulos, how='symmetric_difference')
ax = diferencia_sim.plot(alpha=0.5, cmap='rainbow')
# Graficamos los contornos de las geometrías originales
df_cuadrados.boundary.plot(ax=ax, color='black')
df_triangulos.boundary.plot(ax=ax, color='black')
```

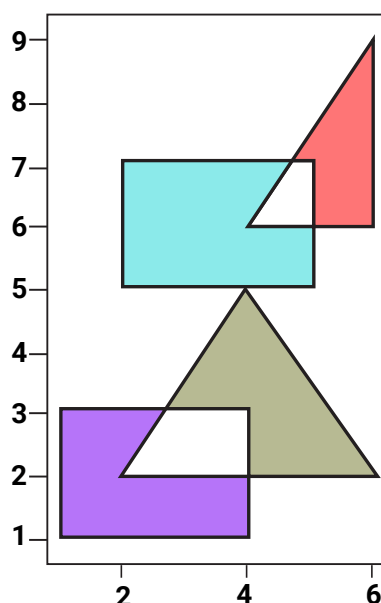


Figura 6. Visualización de una diferencia simétrica de GeoDataFrames.

El resultado de esta operación genera cuatro geometrías distintas, representadas con los colores morado, amarillo, turquesa y rojo. Como podemos observar, todas las intersecciones de ambos conjuntos han sido “recortadas”.

Para facilitar la interpretación del resultado, hemos añadido el contorno de las geometrías originales en la gráfica. Es importante recordar que las áreas en blanco no forman parte del resultado de la diferencia simétrica.

■ Ejemplo: Zonas Urbanas

Ahora que conoces las cuatro operaciones de conjuntos básicas, es momento de explorar cómo aplicarlas para mejorar la visualización de datos en un mapa con información real. Volvamos al ejemplo de los estados de la República Mexicana y analicemos cómo podríamos identificar y visualizar las zonas urbanas.

Para este ejercicio, supondremos que las zonas urbanas se encuentran dentro de un radio de 120 km desde el centro de cada capital estatal. Esta información podría ser útil, por ejemplo, para planificar la distribución de medicamentos o vacunas en las áreas más pobladas.

Utilizaremos los archivos de datos empleados en las lecciones anteriores. Recuerda que, si los archivos están almacenados en tu Google Drive, es necesario montarlo antes de acceder a ellos.

```
# Montamos Google Drive
from google.colab import drive
drive.mount('/content/gdrive')

# Cargamos los datos de los estados
estados = geopandas.read_file('/content/gdrive/MyDrive/mexican-states.shp')
# Cargamos los datos de las capitales
df = pd.read_csv('/content/gdrive/MyDrive/mexican-capitals.csv')
```

```

capitales = geopandas.GeoDataFrame(df, geometry=geopandas.
points_from_xy(df.Long, df.Lat))

# Aseguramos que sea el mismo CRS y que el CRS use metros
como medida de distancia
capitales = capitales.set_crs(estados.crs)
estados = estados.to_crs(epsg=3395)
capitales = capitales.to_crs(epsg=3395)

# Pintamos el mapa base
mapa = estados.plot(alpha=0.5, cmap="rainbow")
# Pintamos los puntos correspondientes a las capitales del
país
capitales.plot(ax=mapa, marker='v', color='gray',
markersize=20)

```

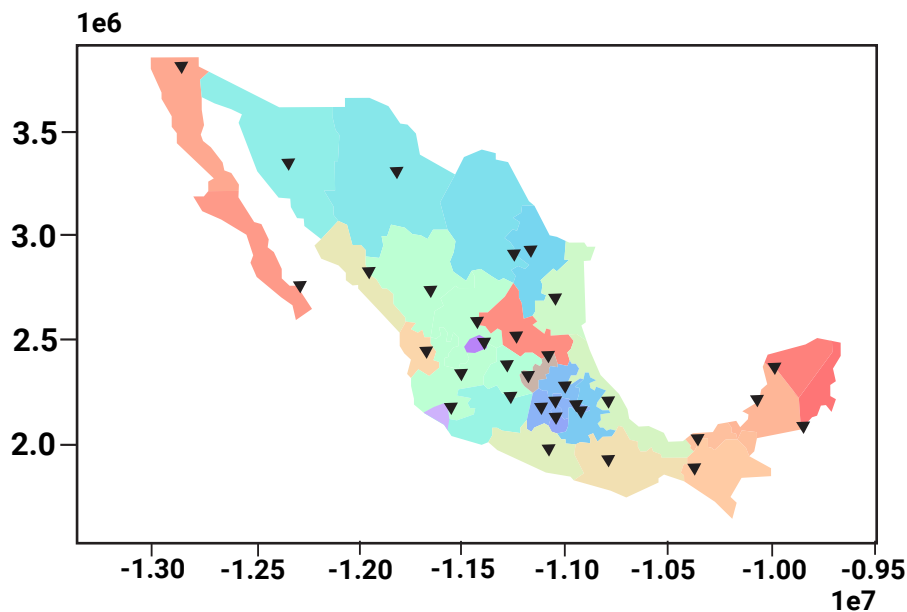


Figura 7. Ejemplo de visualización de estados y capitales de la República Mexicana.

Nota: Si al utilizar la función `to_crs` obtienes el siguiente error:

`ValueError: Cannot transform naive geometries. Please set a crs on the object first.`

Significa que la geometría aún no tenía asignado un `crs`, por lo que debes usar primero un `set_crs`.

Hasta este punto, hemos cargado dos `GeoDataFrames` y generado un mapa en el que cada estado tiene un color distinto, mientras que los centros de las capitales están marcados con un triángulo gris. Este mapa ya nos proporciona una idea general de las zonas urbanas en México, pero podemos hacerlo aún más claro aplicando una de las operaciones que acabamos de aprender.

¿Cuál crees que sería la mejor opción para delimitar las zonas urbanas en un radio de 120 km desde el centro de las capitales?

Para lograrlo, aplicaremos una intersección entre el mapa de los estados y los círculos generados alrededor de cada capital con un radio de 120 km. El resultado nos mostrará las zonas urbanas (o al menos las más cercanas a las capitales). Además, las zonas donde se superponen varios círculos indicarán regiones densamente pobladas que están cerca de múltiples capitales.

```
# Lo primero es crear un buffer de 120 km (el radio que estamos buscando)
```

```
capitales['geometry'] = capitales.buffer(120000)
```

```
# Ahora graficamos solamente la intersección de los estados con  
# los círculos creados a partir de la localización de las capitales
```

```
zonas_urbanas = geopandas.overlay(estados, capitales,  
how='intersection', keep_geom_type=False)
```

```
#ax = estados.boundary.plot(color='gray')
```

```
zonas_urbanas.plot(alpha=0.5, edgecolor='k', cmap="rainbow")
```

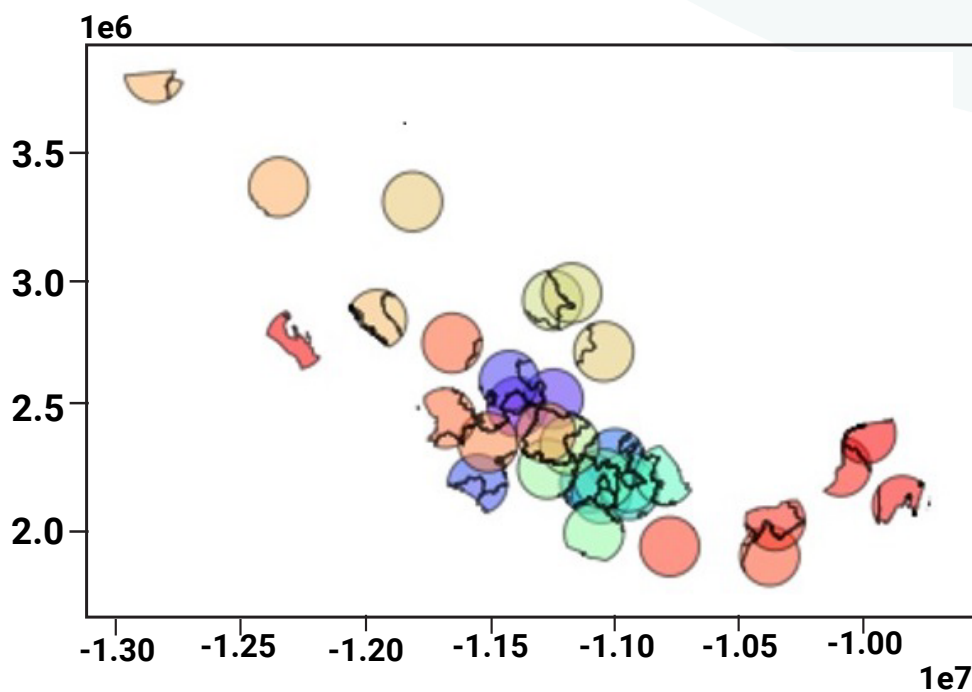


Figura 8. Visualización de la intersección entre estados y capitales de la República Mexicana.

Ahora bien, ¿qué pasaría si, en lugar de graficar la intersección, utilizáramos la diferencia? En ese caso, obtendríamos las zonas más alejadas de las capitales, lo que podría ser útil para otros análisis, como la identificación de áreas con menor acceso a servicios.

¿Se te ocurre alguna aplicación para esta visualización? Mientras reflexionas al respecto, te mostraremos el mapa resultante. Quizás, al verlo, se te ocurran nuevas ideas sobre cómo aprovechar esta información.

```
# Ahora graficamos solamente la diferencia de los estados con
# los círculos creados a partir de la localización de las
capitales
zonas_urbanas = geopandas.overlay(estados, capitales,
how='difference', keep_geom_type=False)

#ax = estados.boundary.plot(color='gray')
zonas_urbanas.plot(alpha=0.5, edgecolor='k', cmap="rainbow")
```

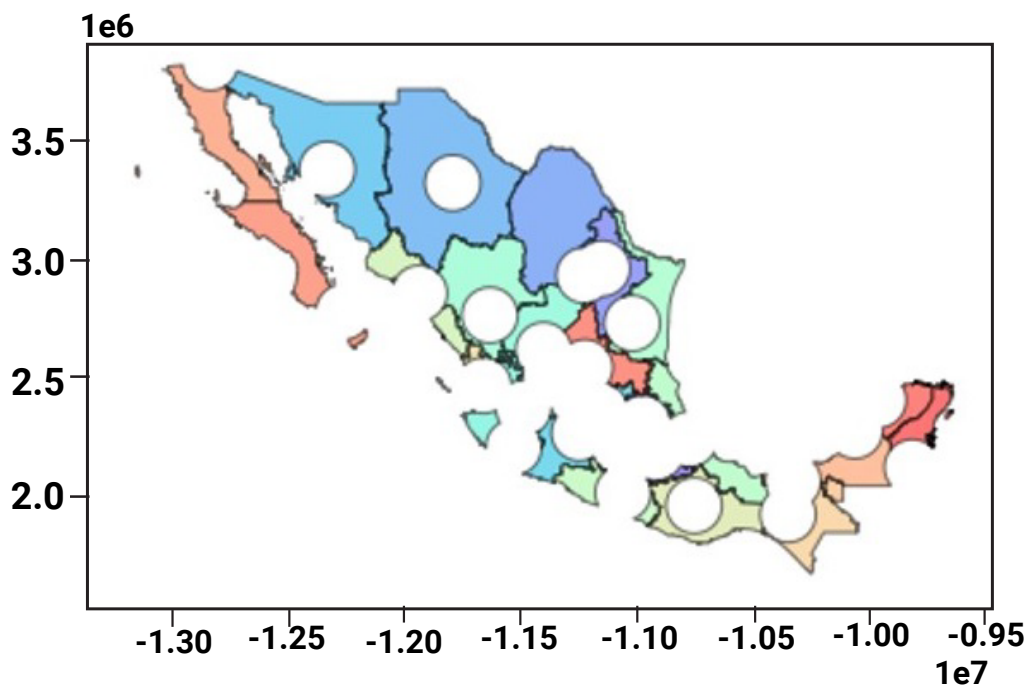


Figura 9. Visualización de una diferencia entre los estados y capitales de la República Mexicana.

Antes de continuar...

En este subtema exploraste distintas operaciones que pueden aplicarse a dos GeoDataFrames y descubriste algunas de sus aplicaciones prácticas. A su vez, comprendiste la importancia de elegir la operación adecuada para representar correctamente la información en una visualización geoespacial.

Ahora, tómate un momento para reflexionar sobre lo aprendido:

- ¿Qué tipo de visualización requiere el uso de cada operación?
- ¿Consideras que la unión y la diferencia son operaciones opuestas?
- ¿Crees que la intersección y la diferencia simétrica son operaciones opuestas?

Para poner en práctica lo aprendido, durante la próxima semana busca información geoespacial disponible en tu entorno de trabajo. Analiza los diferentes tipos de datos que encuentres y reflexiona sobre lo siguiente:

- ¿Existe alguna relación entre los distintos conjuntos de datos espaciales que identificaste?
- ¿Podrías aplicar alguna de las operaciones vistas en este subtema para visualizar estas relaciones de manera más clara y efectiva?

Esta reflexión te ayudará a conectar los conceptos aprendidos con casos reales y a mejorar tu capacidad para analizar y representar datos espaciales.

Fusión de datos

Después de explorar las operaciones de conjuntos más comunes para transformar datos geoespaciales, es momento de aprender a fusionar información para crear conjuntos de datos más completos. La **fusión de datos permite combinar información de distintas fuentes**, enriqueciendo los análisis y mejorando la visualización de los resultados.

En **GeoPandas**, existen dos formas principales de fusionar datos:

- **Unión por atributos (`Attribute Join`):** Combina una `GeoSeries` o `GeoDataFrame` con un `pandas.Series` o `pandas.DataFrame` utilizando una variable común, de manera similar a las uniones en `pandas`.
- **Unión espacial (`Spatial Join`):** Fusiona datos en función de su relación geoespacial, combinando observaciones de dos `GeoSeries` o `GeoDataFrames` según su ubicación en el espacio.

A lo largo de esta sección, exploraremos cómo aplicar estas técnicas para integrar y manejar grandes volúmenes de datos en estudios geoespaciales.

Adjuntar (`Appending`)

La forma más sencilla de unir dos `GeoDataFrames` es mediante la adición de datos, donde la información del segundo `GeoDataFrame` se integra al primero. Esto se logra con la función `concat()` de `pandas`.

Es importante recordar que, para adjuntar columnas de geometría, ambos `GeoDataFrames` deben tener el mismo sistema de referencia de coordenadas (*CRS*, por sus siglas en inglés: *Coordinate Reference System*).

En el siguiente fragmento de código, cargaremos los datos de todos los estados de la República Mexicana y luego guardaremos los estados de la región centro en un GeoDataFrame separado. Dado que todos los datos provienen del mismo conjunto original, podemos asumir que comparten el mismo CRS. Sin embargo, si existe alguna duda, siempre es recomendable verificarlo y, de ser necesario, establecerlo con las funciones `set_crs()` o `to_crs()`.

```
# Cargamos los datos de los estados
estados = geopandas.read_file('/content/gdrive/MyDrive/
mexican-states.shp')
# Esta vez sólo necesitamos algunas columnas
estados = estados.loc[:, ['ISO3166-2', 'name', 'geometry']]
# Separamos cada uno de los siguientes estados
# De esta forma cada uno está representado en un GeoDataFra-
me diferente
hidalgo = estados.iloc[[6]]
tlaxcala = estados.iloc[[2]]
puebla = estados.iloc[[7]]
morelos = estados.iloc[[4]]
mexico = estados.iloc[[5]]
cdmx = estados.iloc[[3]]
```

Ahora uniremos los datos de cada estado para obtener un único GeoDataFrame que contenga la información de todos los estados del centro de México.

Finalmente, visualizaremos tanto los datos de cada uno de los estados como su representación en el mapa.

```
# Ahora vamos a adjuntar la información de estos 6 GeoData-
Frames
# para crear un GeoDataFrame que contenga todos los estados
# de la parte central de la República Mexicana
mx_centro = pd.concat([hidalgo, tlaxcala, puebla, morelos,
mexico, cdmx])
mx_centro
```

	ISO3166-2	name	geometry
6	MX-HID	Hidalgo	POLYGON ((-99.73527 20.56486, -99.73525 20.564...
2	MX-TLA	Tlaxcala	POLYGON ((-98.5415 19.46082, -98.54158 19.4607...
7	MX-PUE	Puebla	POLYGON ((-97.26671 19.38249, -97.26549 19.381...
4	MX-MOR	Morelos	POLYGON ((-98.90339 19.03848, -98.90061 19.038...
5	MX-MEX	México	POLYGON ((-100.13794 20.04431, -100.13759 20.0...
3	MX-CMX	Ciudad de México	POLYGON ((-99.2787 19.1317, -99.28259 19.13821...

Figura 10. Visualización de los datos de los GeoDataFrames adjuntos.

```
# Vamos a graficar el nuevo conjunto de datos
mx_centro.plot(cmap='BuPu_r')
```

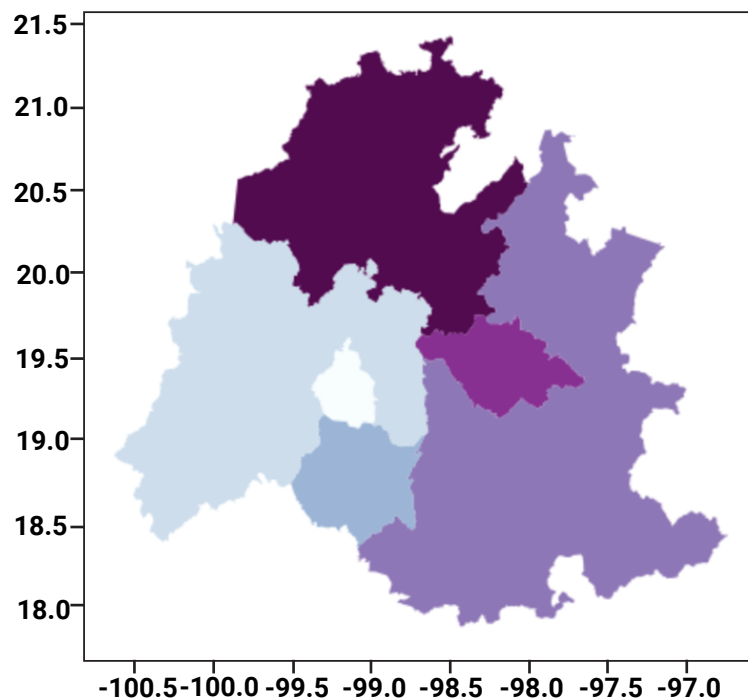


Figura 11. Visualización en el mapa de los GeoDataFrames adjuntos.

Unión por Atributos (Attribute Join)

El segundo método de fusión de datos que exploraremos es la **unión por atributos**, que permite combinar un GeoDataFrame con otro GeoDataFrame o con un DataFrame de pandas utilizando una variable en común. Esta operación funciona de manera similar a `merge` o `join` en pandas.

Es fundamental considerar que, si el argumento de la izquierda es un GeoDataFrame, el resultado conservará su estructura geoespacial. Sin embargo, si el argumento de la izquierda es un DataFrame, el resultado perderá la información de geometría y ya no será un GeoDataFrame.

Para ilustrar este concepto, utilizaremos el mismo código que carga los datos de todos los estados de la República Mexicana, aplicando un filtro para seleccionar sólo algunas columnas. Luego, cargaremos un archivo con información sobre las capitales de cada estado, el cual comparte la variable `name` con el conjunto de datos de los estados.

Es importante notar que:

- `estados` es un GeoDataFrame, mientras que `df` es un DataFrame.
- La unión se realiza mediante la función `merge()`, utilizando la variable `name` como clave, ya que está presente en ambos conjuntos de datos.
- Al mantener el GeoDataFrame en el lado izquierdo de la operación, aseguramos que el resultado también sea un GeoDataFrame, lo que nos permitirá graficarlo en un mapa posteriormente.

```
# Cargamos los datos de los estados
estados = geopandas.read_file('/content/gdrive/MyDrive/
mexican-states.shp')
# Esta vez sólo necesitamos algunas columnas
estados = estados.loc[:, ['ISO3166-2', 'name', 'geometry']]
# Cargamos los datos de las capitales
df = pd.read_csv('/content/gdrive/MyDrive/mexican-capitals-
by-name.csv')
# También usamos sólo algunas columnas
df = df.loc[:, ['name', 'capital']]
# Unimos el GeoDataFrame con el DataFrame
# nota como estados está del lado izquierdo del merge
estados = estados.merge(df, on='name')
estados.head()
```

	ISO3166-2	name	geometry	capital
0	MX-AGU	Aguascalientes	POLYGON ((-102.74530 21.72005, -102.74859 21.7...	Aguascalientes
1	MX-COL	Colima	MULTIPOLYGON (((-103.48859 18.96489, -103.4884...	Colima
2	MX-TLA	Tlaxcala	POLYGON ((-98.54150 19.46082, -98.54158 19.460... Tlaxcala de Xicohténcatl	
3	MX-CMX	Ciudad de México	POLYGON ((-99.27870 19.13170, -99.28259 19.138...	Ciudad de México
4	MX-MOR	Morelos	POLYGON ((-98.90339 19.03848, -98.90061 19.038...	Cuernavaca

Figura 12. Visualización de los datos del GeoDataFrame resultante de la unión de atributos.

```
# Ahora vamos a graficar el nuevo conjunto de datos
estados.plot(cmap='Greens')
```

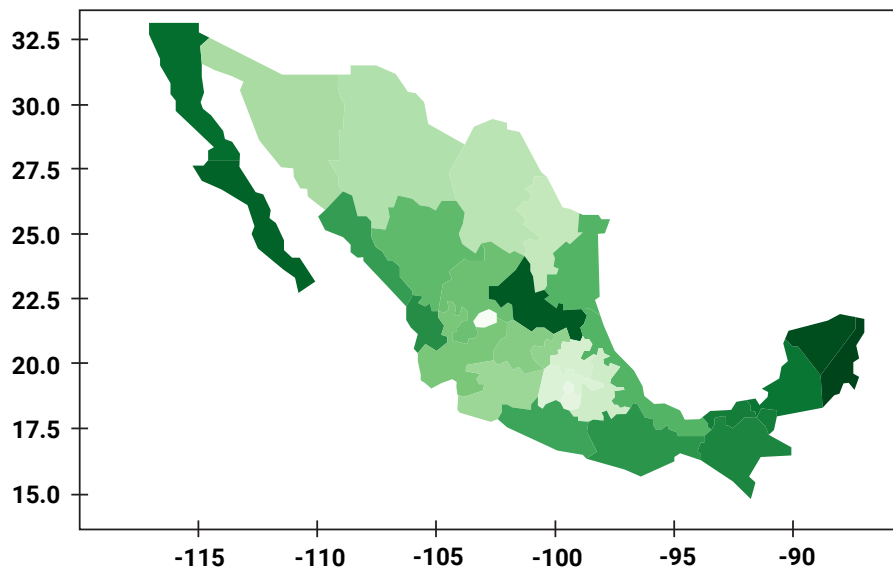


Figura 13. Visualización en el mapa del GeoDataFrame resultante de la unión de atributos.

Unión Espacial (Spatial Join)

El segundo método de fusión de datos que exploraremos es la **unión espacial**, que permite combinar dos GeoSeries o dos GeoDataFrames en función de la relación espacial entre sus geometrías.

GeoPandas permite realizar uniones espaciales a través de dos funciones principales:

- **sjoin()**: Une los datos basándose en relaciones geométricas (intersección, contención, etc.).
- **sjoin_nearest()**: Une los datos en función de la proximidad, con la opción de establecer un radio de búsqueda máximo.

Para ilustrar su uso, continuaremos con el código que carga los datos de todos los estados de la República Mexicana, aplicando un filtro para seleccionar sólo algunas columnas. Además, trabajaremos con un GeoDataFrame que contiene los polígonos que definen las distintas regiones del país. A continuación, presentamos ambos conjuntos de datos y su estructura.

```
# Cargamos los datos de los estados
estados = geopandas.read_file('/content/gdrive/MyDrive/TLG-
Geovisualizacion/NOTEBOOKS/mexican-states.shp' )
# Usamos sólo algunas columnas
estados = estados.loc[:, ['ISO3166-2', 'name', 'geometry']]

# Asignamos su region a cada estado
estados['region'] = ['centro oeste', 'centro
                    oeste', 'centro', 'centro', 'centro',
                    'centro', 'centro', 'centro', 'noreste',
                    'noreste', 'noreste', 'noroeste', 'centro
                    oeste', 'centro oeste', 'centro oeste',
                    'centro oeste', 'centro
                    oeste', 'noreste', 'noreste', 'sureste',
                    'sureste', 'noroeste', 'sureste', 'centro
                    oeste', 'sureste',

# Imprimimos el header de los estados
estados.head()

# Disolvemos por regiones
regiones = estados.dissolve(by='region')
# Usamos sólo los datos que vamos a necesitar
regiones = regiones.loc[:, 'geometry']

# GeoDataFrame que contiene las regiones de la República
Mexicana
regiones_gdf = geopandas.GeoDataFrame(geopandas.
GeoSeries(regiones))
```



```
# Imprimimos el GeoDataFrame con las regiones
regiones_gdf.head()

# Graficamos las regiones de México
regiones_gdf.plot(alpha=0.5, cmap='rainbow')
```

	ISO3166-2	name	geometry	region
0	MX-AGU	Aguascalientes	MULTIPOLYGON (((-102.7453 21.72005, -102.74859...	centro oeste
1	MX-COL	Colima	MULTIPOLYGON (((-103.48859 18.96489, -103.4884...	centro oeste
2	MX-TLA	Tlaxcala	POLYGON ((-98.5415 19.46082, -98.54158 19.4607...	centro
3	MX-CMX	Ciudad de México	POLYGON ((-99.2787 19.1317, -99.28259 19.13821...	centro
4	MX-MOR	Morelos	POLYGON ((-98.90339 19.03848, -98.90061 19.038...	centro

Figura 14. Ejemplo del contenido del GeoDataFrame de los estados.

region	
centro	POLYGON ((-99.06596 18.33265, -99.06612 18.333...
centro oeste	MULTIPOLYGON (((-106.24013 21.33311, -106.2398...
noreste	POLYGON ((-104.24942 22.7484, -104.24981 22.74...
noroeste	MULTIPOLYGON (((-114.2928 27.11006, -114.29281...
sureste	MULTIPOLYGON (((-96.05972 15.77985, -96.06001 ...

Figura 15. Contenido del GeoDataFrame de las regiones de México.

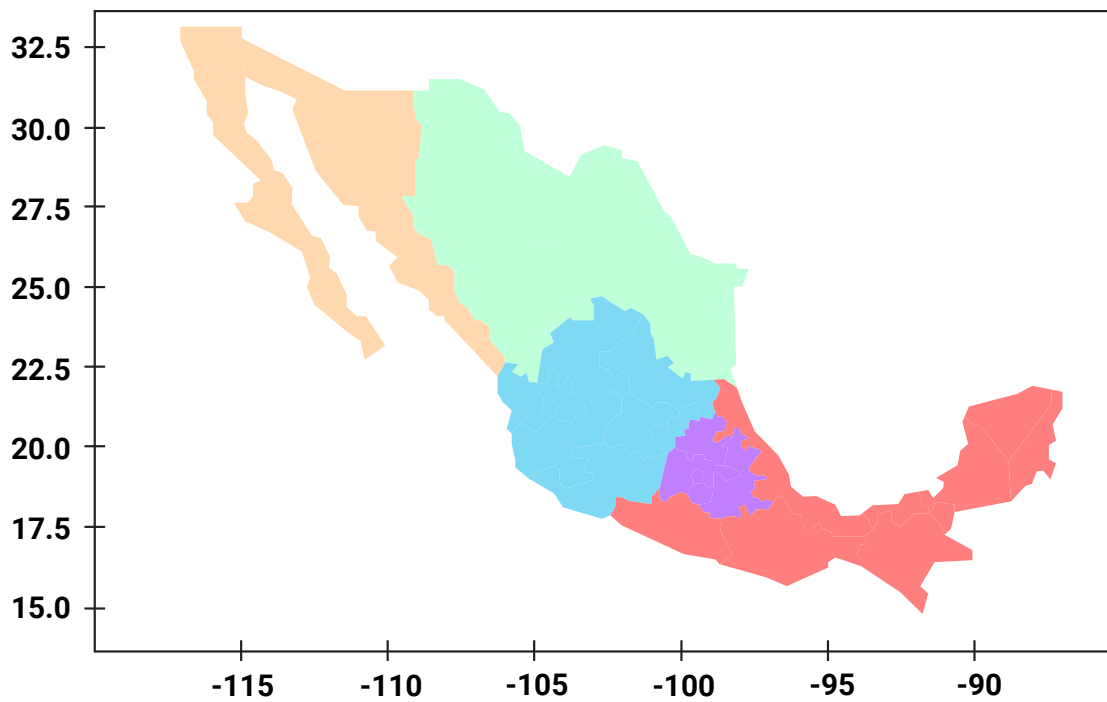


Figura 16. Visualización en el mapa del GeoDataFrame de las regiones del país.

Una vez preparados los datos, realizaremos la **unión espacial** entre el GeoDataFrame de los estados y el de las regiones utilizando la función `sjoin`. Esto nos permitirá agregar la información de la región correspondiente a cada estado, basándonos en la intersección entre ambos conjuntos de datos.

```
# Hacemos el join de los estados y las regiones
estados_por_regiones = geopandas.sjoin(estados, regiones_gdf,
how='inner', predicate='intersects')
estados_por_regiones.head()
```

Graficamos nuevamente para comprobar que es un GeoDataFrame
estados_por regiones.plot(alpha=0.5, cmap='rainbow')

	ISO3166-2	name	geometry	region_left	region_right
0	MX-AGU	Aguascalientes	MULTIPOLYGON (((-102.7453 21.72005, -102.74859...	centro oeste	centro oeste
1	MX-COL	Colima	MULTIPOLYGON (((-103.48859 18.96489, -103.4884...	centro oeste	centro oeste
2	MX-TLA	Tlaxcala	POLYGON ((-98.5415 19.46082, -98.54158 19.4607...	centro	centro
3	MX-CMX	Ciudad de México	POLYGON ((-99.2787 19.1317, -99.28259 19.13821...	centro	centro
4	MX-MOR	Morelos	POLYGON ((-98.90339 19.03848, -98.90061 19.038...	centro	sureste

Figura 17. Visualización de la unión espacial de dos GeoDataFrames.

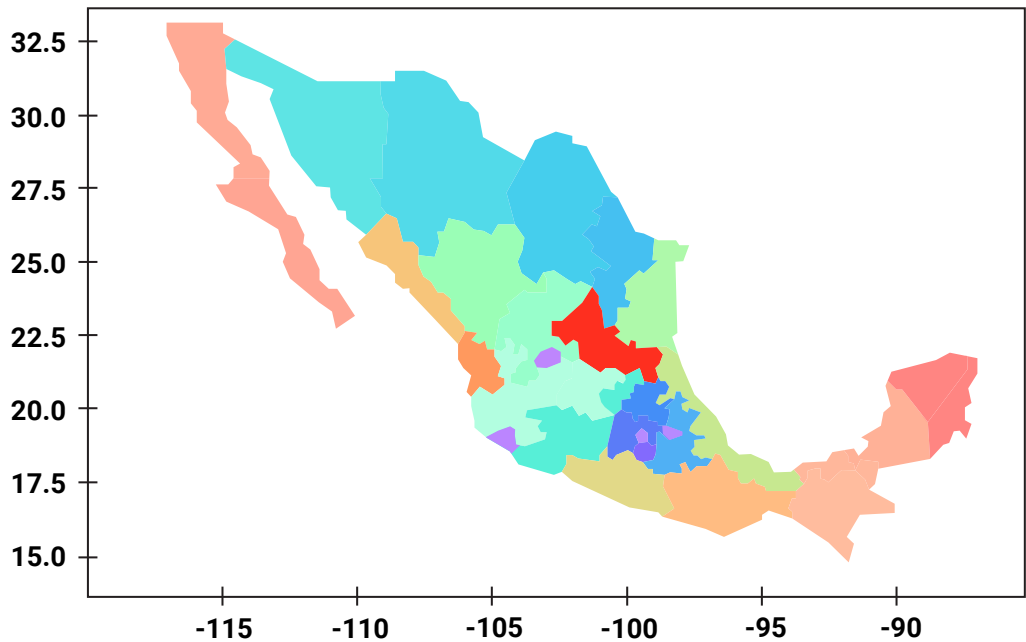


Figura 18. Visualización del mapa después de la unión espacial de dos GeoDataFrames.

La función `sjoin` recibe dos parámetros clave:

- **predicate**: Define la relación geométrica que determina si dos objetos deben unirse.
- **how**: Especifica el tipo de unión a realizar y qué geometría se conservará en el GeoDataFrame resultante.

En las siguientes tablas, se detallan las opciones disponibles para ambos parámetros.

Atributo <code>predicate</code>	
Valor	Descripción
<code>intersects</code>	Se realiza la unión si las geometrías se intersectan.
<code>contains</code>	Se realiza la unión si la primera geometría contiene a la segunda.
<code>within</code>	Se realiza la unión si la primera geometría está totalmente contenida en la segunda.
<code>touches</code>	Se realiza la unión si las geometrías se tocan.
<code>crosses</code>	Se realiza la unión si las geometrías se cruzan.
<code>overlaps</code>	Se realiza la unión si las geometrías se empalman.

Tabla 1. Posibles valores para el atributo `predicate` de la función `sjoin`.

Atributo <code>how</code>	
Valor	Descripción
<code>left</code>	Usa los índices del primer GeoDataFrame y retiene su geometría en el resultado.
<code>right</code>	Usa los índices del segundo GeoDataFrame y retiene su geometría en el resultado.
<code>inner</code>	Usa la intersección de los índices de ambos GeoDataFrames y retiene la geometría del primero en el resultado.

Tabla 2. Posibles valores para el atributo `how` de la función `sjoin`.

Para analizar relaciones espaciales más complejas, se pueden combinar operaciones geométricas con `sjoin`. Por ejemplo, para encontrar polígonos dentro de una cierta distancia de un punto, se puede utilizar `buffer` para expandir el punto en un círculo y luego realizar la intersección con los polígonos.

Por otra parte, la función `sjoin_nearest` permite unir datos basándose en la distancia más corta entre geometrías. Comparte el argumento `how` con `sjoin` e incorpora dos parámetros adicionales:

- `max_distance`: Define un radio máximo de búsqueda para encontrar la geometría más cercana. Su uso puede mejorar el rendimiento.
- `distance_col`: Si se especifica, el GeoDataFrame resultante incluirá una columna con este nombre que almacenará la distancia calculada entre las geometrías coincidentes.

Estas técnicas de unión espacial son fundamentales para integrar y analizar información geoespacial en estudios avanzados. Si quieres más información sobre los tipos de fusión de datos disponibles en GeoPandas, puedes consultar la referencia correspondiente al final de esta lección.

Antes de terminar...

En este subtema exploraste cómo fusionar datos entre dos GeoDataFrames o entre un GeoDataFrame y un DataFrame convencional. Ahora, tómate un momento para reflexionar sobre lo aprendido:

- ¿Has encontrado similitudes entre estas técnicas y ejercicios previos que hayas realizado?
- ¿Crees que la fusión de datos puede mejorar la visualización de información geoespacial?
- ¿Se te ocurre algún conjunto de datos geoespaciales que te gustaría combinar para obtener nueva información o mejorar su análisis?

Durante la próxima semana, analiza los datos geoespaciales de tu empresa u organización e identifica lo siguiente:

- ¿Podrías extraer información valiosa combinando dos GeoDataFrames?
- ¿Alguna de estas técnicas permitiría representar mejor los datos en un mapa?
- ¿La adición de más información facilitaría el análisis o podría dificultar la visualización?

Material adicional

Geopandas. (s.f.) *Merging data*. [Fusión de datos]. Geopandas. Recuperado el 17 de abril de 2025 de: https://geopandas.org/docs/user_guide/mergingdata.html

Geopandas. (s.f.) *Set operations with overlay*. [Operaciones de conjunto con superposición]. Geopandas. Recuperado el 17 de abril de 2025 de: https://geopandas.org/en/stable/docs/user_guide/set_operations.html

Matplotlib. (s.f.) *Choosing Colormaps*. [Elegiendo de mapas de color]. Matplotlib. Recuperado el 17 de abril de 2025 de: <https://matplotlib.org/2.0.2/users/colormaps.html>

Nota: Los enlaces a las páginas web incluidos en este Material adicional fueron consultados y verificados por última vez el 17 de abril de 2025 y su disponibilidad depende del autor.

Glosario

- **CRS (*Coordinate Reference System*)**: Sistema de referencia que define cómo se interpretan las coordenadas de una geometría en el espacio.
- **Diferencia (*Difference*)**: Operación que devuelve las partes de un conjunto que no se superponen con el otro.
- **Diferencia Simétrica (*symmetric difference*)**: Operación que elimina las intersecciones entre dos conjuntos, dejando sólo las partes exclusivas de cada uno.
- **Fusión de Datos (*Data Fusion*)**: Proceso de combinar información de diferentes fuentes para obtener un conjunto de datos más completo.
- **GeoDataFrame**: Estructura de datos en GeoPandas que extiende un DataFrame de pandas para incluir información geoespacial.
- **Geometría**: Representación de un objeto espacial en un GeoDataFrame, como puntos, líneas o polígonos.
- **Intersección (*intersection*)**: Operación que devuelve las partes en común entre dos conjuntos de datos geoespaciales.
- **Unión (*Union*)**: Operación que combina todos los elementos de dos conjuntos geoespaciales en un sólo conjunto.
- **Unión Espacial (*Spatial Join*)**: Método que fusiona datos geoespaciales en función de su relación espacial, como intersección o proximidad.
- **Unión por Atributos (*Attribute Join*)**: Método que combina un GeoDataFrame con otro DataFrame usando una columna en común, sin considerar la geometría.

Bibliografía

Geopandas. (s.f.). *geopandas.GeoDataFrame*. Geopandas. Recuperado el 17 de abril de 2025 de: <https://geopandas.org/en/stable/docs/reference/api/geopandas.GeoDataFrame.html>

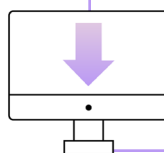
Geopandas. (s.f.). *Introduction to GeoPandas*. [Introducción a GeoPandas]. Geopandas. Recuperado el 17 de abril de 2025 de: https://geopandas.org/en/stable/getting_started/introduction.html

Geopandas. (s.f.). *Merging data*. [Fusión de datos]. Geopandas. Recuperado el 17 de abril de 2025 de: https://geopandas.org/docs/user_guide/mergingdata.html

Geopandas. (s.f.). *Set operations with overlay*. [Operaciones de conjunto con superposición]. Geopandas. Recuperado el 17 de abril de 2025 de: https://geopandas.org/en/stable/docs/user_guide/set_operations.html

Wasser, L. (03 de septiembre de 2019). *Lesson 3. Coordinate Reference System and Spatial Projection*. [Lección 3. Sistema de referencia de coordenadas y proyección espacial]. EarthLab. Recuperado el 17 de abril de 2025 de: <https://www.earthdatascience.org/courses/earth-analytics/spatial-data-r/intro-to-coordinate-reference-systems/>

Nota: Los enlaces a las páginas web incluidos en esta bibliografía fueron consultados y verificados por última vez el 17 de abril de 2025 y su disponibilidad depende del autor.



Descarga este documento en la sección “**Archivos adjuntos**” de la plataforma.